Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Сравнение сигмоиды и ReLU на скрытом слое многослойного персептрона»

Выполнил:
Студент 4 курса
Группы ИИ-26
Турич Д. А.
Проверил:
Никитин С. К.

Брест 2026

Цель работы

Проверить влияние сигмоиды и ReLU в скрытом слое MLP 2–2–1 на скорость сходимости BCE, устойчивость, способность решить XOR и геометрию разделяющей поверхности.

1. Вариант 6

A	B	A⊕B
-9	-9	-9
-9	6	6
6	-9	6
6	6	-9

2. Модель и обучение

В обеих моделях два входа, два скрытых нейрона, один сигмоидный выход и функция потерь BCE. Метки нормализованы −9→0 и 6→1. Модель A применяет sigmoid на скрытом слое, модель B — ReLU(z)=max(0,z). Backpropagation обновляет параметры после каждого примера. $\eta=0{,}5$; $E_e=0{,}01$; предел 20 000 эпох; пять seed: 6, 7, 11, 14, 18.

Sigmoid: $d\sigma/dz=\sigma(z)(1-\sigma(z))$. ReLU: $dReLU/dz=1$ для $z>0$ и 0 для $z\leq 0$. Для BCE+sigmoid на выходе $\delta_2=\hat{y}-t$. Для скрытого слоя $\delta_1=(\delta_2 W_2^T)\odot f'(z_1)$. «Мёртвый ReLU» на всей таблице обнаруживается как $z_1\leq 0$ для всех четырёх входов; его веса не обновляются через производную ReLU.

3. Пять запусков и метрики

Конфигурация A — Sigmoid

Seed	Эпохи	E_s	Accuracy	MAE	Мёртвые ReLU	$E_s \leq E_e$
6	2368	0.00999	100%	0.0374	0	да
7	2792	0.01000	100%	0.0374	0	да
11	20000	1.91012	75%	4.9746	0	нет
14	2523	0.01000	100%	0.0374	0	да
18	9647	0.01000	100%	0.0374	0	да

Критерий достигнут в 4 из 5 запусков. Разброс числа эпох успешных запусков: 2368–9647.

Конфигурация B — ReLU

Seed	Эпохи	E_s	Accuracy	MAE	Мёртвые ReLU	$E_s \leq E_e$
6	370	0.00998	100%	0.0374	0	да
7	376	0.00998	100%	0.0373	0	да
11	20000	1.90964	75%	5.0143	1	нет
14	352	0.00995	100%	0.0373	0	да

Seed	Эпохи	E_s	Accuracy	MAE	Мёртвые ReLU	$E_s \leq E_e$
18	347	0.01000	100%	0.0374	0	да

Критерий достигнут в 4 из 5 запусков. Разброс числа эпох успешных запусков: 347–376.

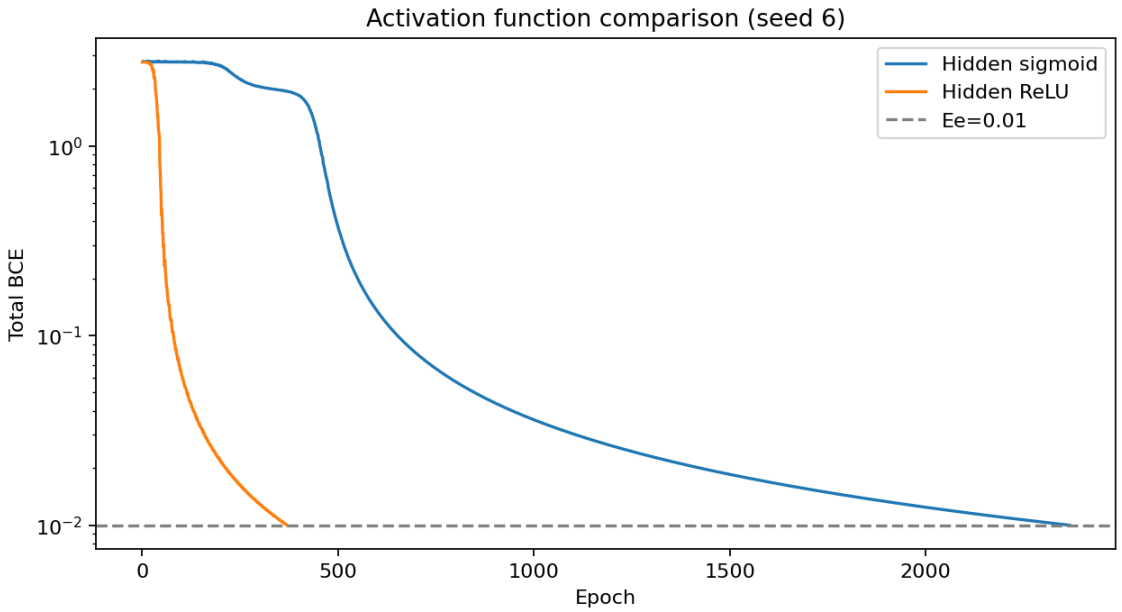


Рисунок 1 — Динамика BCE для обеих функций активации

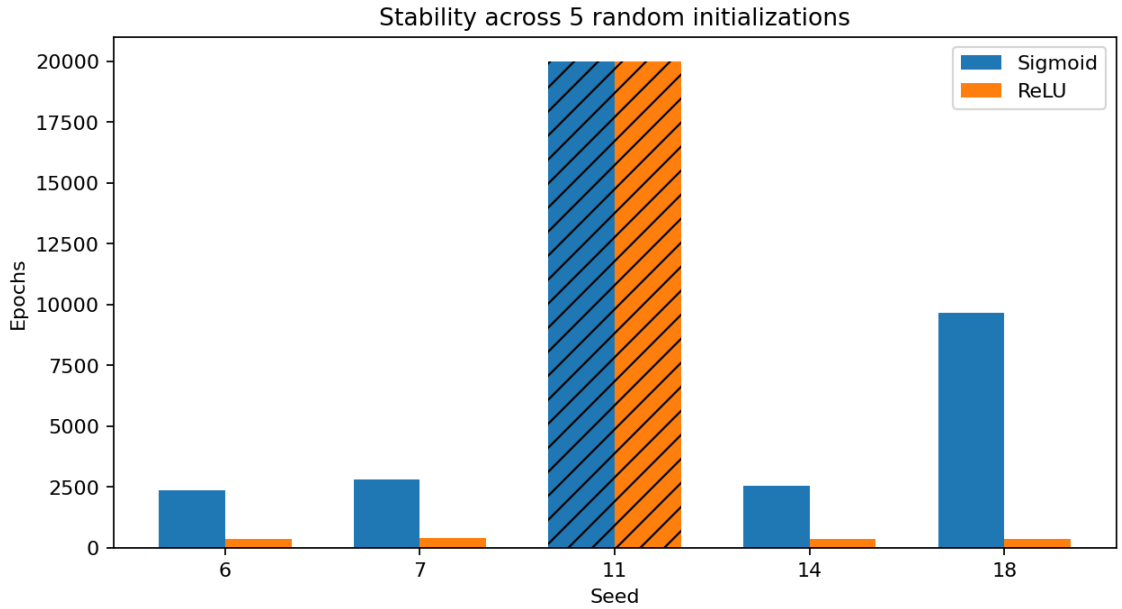


Рисунок 2 — Стабильность сходимости по пяти seed

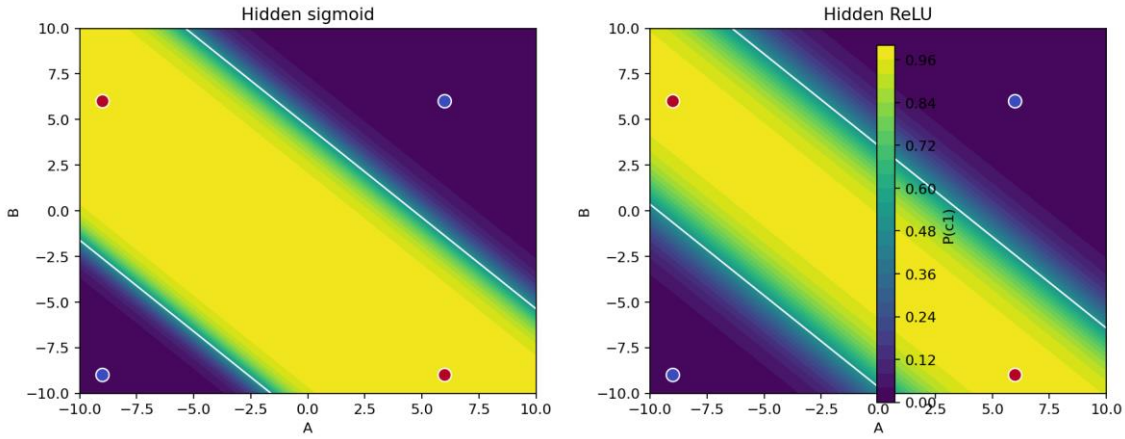


Рисунок 3 — Разделяющие поверхности (Sigmoid / ReLU)

4. Работа в режиме функционирования

A	B	Выход 0..1	Исходная шкала	Класс
-9	-9	0.0009	-8.9860	-9
-9	6	0.9959	5.9388	6
6	-9	0.9959	5.9388	6
6	6	0.0009	-8.9870	-9
-6	4	0.9908	5.8613	6
0	0	0.9524	5.2854	6
6	0	0.1159	-7.2610	-9
-10	10	0.9528	5.2914	6

5. Выводы по вопросам методички

1) На успешных инициализациях ReLU сходилась заметно быстрее: 347–376 эпох против 2368–9647 у sigmoid; обе модели достигли порога в 4 из 5 запусков.

2) Сигмоида даёт гладкое изменение вероятности на плоскости входов; ReLU — кусочно-линейный скрытый отклик, который после выходной сигмоиды образует участки с изменением градиента.

3) ReLU на положительной полуоси не насыщается по производной, что облегчает передачу градиента в глубоких сетях. Здесь выигрыш скорости подтверждён для отдельных seed, но не гарантирован для всех.

4) Обе модели решают XOR в удачных запусках, потому что имеют два скрытых нейрона и нелинейность; сама смена активации не устраняет сложность XOR.

5) Для ReLU в запуске seed=11 появился полностью неактивный на выборке нейрон, сходимость не была достигнута; инициализация и скорость обучения важны.

6) Наличие скрытого слоя остаётся необходимым для стандартной однослойной модели с монотонной активацией вне зависимости от выбора Sigmoid или ReLU.

7) На картах разделяющих поверхностей успешных моделей четыре обучающие вершины попадают в ожидаемые классы; области в промежуточных точках различаются по форме и уровню уверенности.

Листинг программы (полный исходный код)

В архиве код также сохранён отдельными редактируемыми файлами Python; ниже приведён полный листинг без скрытых библиотек машинного обучения.

Файл: xor_engine.py

```
"""MLP 2-2-1 and single layer perceptron. NumPy is the only dependency.
All losses are totals over the four training examples, evaluated after an epoch.
"""
import numpy as np

C0, C1 = -9.0, 6.0
X_RAW = np.array([[-9.,-9.],[-9.,6.],[6.,-9.],[6.,6.]])
Y_RAW = np.array([-9.,6.,6.,-9.])
X = (X_RAW - C0) / (C1-C0)
Y = (Y_RAW - C0) / (C1-C0)

def sigmoid(z):
    z = np.clip(z, -60, 60)
    return 1 / (1+np.exp(-z))

def relu(z):
    return np.maximum(z, 0.)

class MLP:
    def __init__(self, seed=7, activation='sigmoid', loss='bce', lr=1.0):
        rng=np.random.RandomState(seed)
        self.w1=rng.uniform(-.1,.1,(2,2))
        self.b1=rng.uniform(-.1,.1,2)
        self.w2=rng.uniform(-.1,.1,2)
        self.b2=float(rng.uniform(-.1,.1))
        self.activation,self.loss,self.lr=activation,loss,lr

    def forward(self, x):
        z1=x@self.w1+self.b1
        h=sigmoid(z1) if self.activation=='sigmoid' else relu(z1)
        p=sigmoid(h@self.w2+self.b2)
        return z1,h,p

    def predict(self, x):
        return np.asarray(self.forward(np.asarray(x, dtype=float))[2])

    def train_step(self,x,t):
        z,h,p=self.forward(x)
        # BCE + sigmoid: dL/dz = p-t; MSE=1/2*(p-t)^2.
        delta=(p-t) if self.loss=='bce' else (p-t)*p*(1-p)
        dh=(self.w2*delta)*(h*(1-h) if self.activation=='sigmoid' else (z>0))
        old_w2=self.w2.copy()
        self.w2-=self.lr*delta*h
        self.b2-=self.lr*delta
        self.w1-=self.lr*np.outer(x,dh)
        self.b1-=self.lr*dh

    def total_loss(self, xx, yy):
        p=np.clip(self.predict(xx),1e-12,1-1e-12)
        if self.loss=='bce': return float(-np.sum(yy*np.log(p)+(1-yy)*np.log1p(-p)))
        return float(.5*np.sum((p-yy)**2))

    def fit(self,xx,yy,max_epochs=20000,ee=.01,shuffle=True):
        rng=np.random.RandomState(42)
        history=[]
        for epoch in range(1,max_epochs+1):
            order=rng.permutation(len(xx)) if shuffle else range(len(xx))
            for i in order:self.train_step(xx[i],yy[i])
            error=self.total_loss(xx,yy)
            history.append(error)
            if error<=ee:break
        return dict(epochs=epoch,Es=error,converged=error<=ee,history=np.asarray(history))

class SinglePerceptron:
    def __init__(self,seed=7,lr=1.):
        rng=np.random.RandomState(seed)
        self.w=rng.uniform(-.1,.1,2)
        self.b=float(rng.uniform(-.1,.1))
        self.lr=lr
    def predict(self,xx):return sigmoid(np.asarray(xx)@self.w+self.b)
    def fit(self,xx,yy,max_epochs=20000,ee=.01):
```

```
072:         rng=np.random.RandomState(42);h=[]
073:         for ep in range(1,max_epochs+1):
074:             for i in rng.permutation(len(xx)):
075:                 x,t=xx[i],yy[i]
076:                 p=float(self.predict(x)); delta=(p-t)*p*(1-p)
077:                 self.w-=self.lr*delta*x;self.b-=self.lr*delta
078:             p=self.predict(xx);es=float(.5*np.sum((p-yy)**2));h.append(es)
079:             if es<=ee:break
080:         return dict(epochs=ep,Es=es,converged=es<=ee,history=np.asarray(h))
081:
082: def stats(model, raw_output=False):
083:     out=model.predict(X)
084:     if raw_output:
085:         yr=out;cls=np.where(abs(yr-C0)<=abs(yr-C1),C0,C1)
086:     else:
087:         yr=C0+(C1-C0)*out;cls=np.where(out<.5,C0,C1)
088:     return {'accuracy':float(np.mean(cls==Y_RAW)), 'mae':float(np.mean(abs(yr-Y_RAW))),
089:             'pred_raw':np.asarray(yr).tolist(),'pred_norm':np.asarray(out).tolist(),
090:             'pred_class':np.asarray(cls).tolist()}
091:
092: def inference(model,a,b,raw_output=False):
093:     if not (-10<=a<=10 and -10<=b<=10):raise ValueError('A and B must be in [-10,10]')
094:     pn=float(model.predict((np.array([a,b])-C0)/(C1-C0)))
095:     val=pn if raw_output else C0+15*pn
096:     cls=C0 if abs(val-C0)<=abs(val-C1) else C1
097:     return {'A':a,'B':b,'output_norm':pn,'output_raw':val,'class':cls}
098:
099: TEST_PAIRS=[(-9,-9),(-9,6),(6,-9),(6,6),(-6,4),(0,0),(6,0),(-10,10)]
```

Файл: main.py

```
001: """Lab 3 (variant 6). Compare sigmoid and ReLU hidden activation."""
002: from pathlib import Path
003: import argparse,json
004: import numpy as np
005: import matplotlib.pyplot as plt
006: from xor_engine import MLP,X,Y,X_RAW,Y_RAW,stats,inference
007: SEEDS=[6,7,11,14,18]
008:
009: def run(activation,max_epochs):
010:     runs=[]
011:     for seed in SEEDS:
012:         m=MLP(seed=seed,activation=activation,loss='bce',lr=.5)
013:         r=m.fit(X,Y,ee=.01,max_epochs=max_epochs)
014:         s=stats(m)
015:         inactive=int(np.all((X@m.w1+m.b1)<=0,axis=0).sum()) if activation=='relu' else 0
016:         print(activation,seed,r['epochs'],r['Es'],s['accuracy'],'dead=',inactive)
017:         runs.append((m,r,s,inactive))
018:     return runs
019:
020: def heatmap(model,name,out):
021:     grid=np.linspace(-10,10,120)
022:     a,b=np.meshgrid(grid,grid)
023:     xy=(np.stack([a.ravel(),b.ravel()],axis=-1)+9)/15
024:     vals=model.predict(xy).reshape(a.shape)
025:     fig,ax=plt.subplots(figsize=(6,5))
026:     img=ax.contourf(a,b,vals,levels=25,vmin=0,vmax=1)
027:     ax.scatter(X_RAW[:,0],X_RAW[:,1],c=(Y_RAW==6).astype(int),edgecolor='black',s=70)
028:     ax.set(xlabel='A',ylabel='B',title=name);fig.colorbar(img,ax=ax,label='P(c1)')
029:     fig.tight_layout();fig.savefig(out,dpi=160);plt.close(fig)
030:
031: def main(max_epochs=20000,interactive=False):
032:     res={'sigmoid':run('sigmoid',max_epochs),'relu':run('relu',max_epochs)}
033:     d=Path(__file__).parent/'results';d.mkdir(exist_ok=True)
034:     fig,ax=plt.subplots(figsize=(7,4.5))
035:     for name,runs in res.items():ax.plot(runs[0][1]['history'],label=name)
036:     ax.set_yscale('log');ax.axhline(.01,linestyle='--',label='Ee')
037:     ax.set(xlabel='Epoch',ylabel='Total BCE',title='Activation comparison')
038:
ax.legend();fig.tight_layout();fig.savefig(d/'convergence_runtime.png',dpi=150);plt.close(fig)
039:     for name,runs in res.items():heatmap(runs[0][0],name,d/f'heatmap_{name}_runtime.png')
040:     summary={name:[{'seed':SEEDS[i],'epochs':r['epochs'],'Es':r['Es'],
041:       'converged':r['converged'],**s,'dead_neurons':dead}
042:       for i,(_,r,s,dead) in enumerate(runs)] for name,runs in res.items()}
043:     with open(d/'latest_run.json','w',encoding='utf8') as
f:json.dump(summary,f,ensure_ascii=False,indent=2)
044:     if interactive:
045:         while True:
046:             a=input('A (or q): ').strip()
```

```
            if a.lower()=='q':break
            try:print(inference(res['relu'][0][0],float(a),float(input('B: '))))
            except ValueError as e:print(e)

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--quick',action='store_true');p.add_argument('--interactive',action='store_true')
    a=p.parse_args();main(1000 if a.quick else 20000,a.interactive)
```